Table of Contents

1. Purpose, Scope & How to Use This Plan 3

1.1 Conventions used in this plan . . . 3

1.2 Sequencing principles . . . 4

1.3 Phase overview . . . 4

2. Phase 0 — Stabilisation Sprint (Weeks 1-2) 5

2.1 WS-0.1 Engineering baseline (effort M) . . . 6

2.2 WS-0.2 KB upload stabilisation (effort S) . . . 6

2.3 WS-0.3 Mobile responsiveness sweep (effort M) . . . 7

2.4 WS-0.4 Model catalogue correction (effort S) . . . 7

2.5 Phase 0 summary . . . 8

3. Phase 1 — Metering, Audit & Enforcement Core (Weeks 1-4) 8

3.1 WS-1.1 The usage-event pipeline (effort M) . . . 8

3.2 WS-1.2 Server-side quota enforcement (effort M) . . . 10

3.3 WS-1.3 Plan-by-plan validation matrix (effort S) . . . 11

3.4 WS-1.4 Audit logging of prompts and answers (effort M) . . . 12

3.5 WS-1.5 Streaming responses and chat controls (effort M) . . . 13

3.6 WS-1.6 Organisation dashboard analytics parity (effort S) . . . 14

3.7 Phase 1 summary . . . 14

4. Phase 2 — Answer Quality & Trust (Weeks 5-12) 15

4.1 WS-2.1 Hybrid retrieval and reranking (effort L) . . . 15

4.2 WS-2.2 Inline citations and the sources panel (effort M) . . . 16

4.3 WS-2.3 Conversation memory and controls (effort M) . . . 17

4.4 WS-2.4 Abstention policy and no-answer analytics (effort S) . . . 17

4.5 WS-2.5 Feedback loop and review queue (effort M) . . . 17

4.6 WS-2.6 Voice output (TTS) with per-agent voices (effort M) . . . 18

4.7 WS-2.7 Retrieval playground and chunk inspection (effort M) . . . 18

4.8 WS-2.8 Embedding-model migration and re-index workflow (effort M) . . . 18

4.9 Phase 2 summary . . . 19

5. Phase 3 — Distribution, Agent Intelligence & Enterprise (Months 4-6) 19

5.1 WS-3.1 Embeddable website widget (effort M) . . . 20

5.2 WS-3.2 Tool calling and MCP support (effort L) . . . 20

5.3 WS-3.3 Live knowledge connectors (effort L) . . . 20

5.4 WS-3.4 Evaluation harness (effort M) . . . 20

5.5 WS-3.5 Enterprise pack (effort L) . . . 21

5.6 WS-3.6 Collaboration and activity feed (effort M) . . . 21

5.7 Phase 3 summary . . . 21

6. Shared Engineering Standards 22

6.1 Data-model additions . . . 22

6.2 API conventions . . . 23

6.3 Observability . . . 23

6.4 Security and privacy baseline . . . 24

6.5 Testing pyramid . . . 24

7. Acceptance & QA Matrix 24

8. Risks, Dependencies & Rollout Strategy 26

9. Success Metrics & the Definition of Stable 27

9.1 The definition of stable . . . 27

9.2 Beyond the six months . . . 28

10. References 28

1. Purpose, Scope & How to Use This Plan

This document is the execution companion to the Feature Gap Analysis and AI Enhancement Roadmap. The gap analysis answered the question of what is missing; this plan answers the question of how to build it, in what order, and how to prove each piece is finished. Every one of the 55+ identified gaps and 10 catalogued defects is carried forward here as a workstream with an implementation approach, a data-model impact, and an acceptance test. Nothing from the analysis is dropped, and nothing new is invented: the sole purpose of this document is to convert findings into a stable, sequenced engineering programme that a small full-stack team can execute week by week.

The plan is organised into four phases. Phase 0 is a two-week stabilisation sprint that closes the reported defects and installs the engineering baseline. Phase 1 builds the metering, audit, and enforcement core that every later capability depends on. Phase 2 lifts answer quality and trust: hybrid retrieval, reranking, inline citations, conversation memory, abstention, and the feedback loop. Phase 3 extends distribution and enterprise readiness: the embeddable widget, tool calling and MCP, live connectors, the evaluation harness, and the enterprise pack. Each phase is independently shippable, so the platform improves continuously rather than waiting for a monolithic release.

4	21	10	26
phases, each shippable	tracked workstreams	defects closed by design	weeks to stable, full scope

1.1 Conventions used in this plan

Workstreams are identified as WS-nn within their phase (for example WS-1.3 is the third workstream of Phase 1). Every workstream carries an effort score of S, M, or L, relative to a team of two to four full-stack engineers. Acceptance criteria are written as verifiable statements and collected into the QA matrix in Chapter 7; a workstream is done only when its acceptance criteria pass in a deployed environment, not when the code merges. Code fragments are given as reference implementations: they use Prisma schema syntax and TypeScript-style pseudocode and are meant to be adapted to the actual codebase, not pasted blindly.

The reference stack assumed throughout is the one observable from the product itself: a Next.js full-stack application with TypeScript, Prisma over PostgreSQL, OpenAI-compatible model providers, pgvector or an equivalent vector index for retrieval, and Razorpay for billing. Where a component name is referenced (route handlers, middleware, dashboard components), the names are descriptive and should be mapped onto the real files during Phase 0 codebase orientation. No assumption is made about infrastructure provider; deployment targets are treated as interchangeable.

1.2 Sequencing principles

- **Measurement before capability.** Meters, audit records, and quota enforcement land before any new AI feature, because a capability that cannot be metered cannot be sold, supported, or defended.
- **Trust before reach.** Citations, abstention, and feedback land before distribution channels (widget, tools), because every new channel multiplies the cost of an untrustworthy answer.
- **Server-side enforcement, always.** Plan limits and feature gates are checked in API route handlers and middleware; hiding UI elements is a convenience, never the control itself.
- **Append-only records for billing and audit.** Usage events and audit logs are immutable facts, never updated in place, so disputes and investigations reconstruct history rather than trust a mutable counter.
- **Weekly release train.** Every phase is sliced so that something demonstrable ships each week behind feature flags, keeping risk small and progress visible.

1.3 Phase overview

The table below summarises the four phases: their windows, themes, primary outcomes, and the exit criteria that gate movement to the next phase. The timeline chart in Figure 1 shows the same programme laid out week by week, including the deliberate overlap between Phase 0 and Phase 1, which exists because the stabilisation defects and the metering pipeline touch the same code paths and are best fixed together.

Phase	Window	Theme	Primary outcomes	Exit criteria
0 — Stabilisation	Weeks 1-2	Stop the bleeding	CI and release baseline; KB upload fix; mobile sweep; model catalogue fix	All five reported defects closed or mitigated; CI green on main
1 — Metering & enforcement	Weeks 1-4	You cannot sell what you cannot count	Usage-event pipeline; quota middleware; audit logging; streaming; analytics parity	Meters accurate; per-plan bypass tests pass; audit coverage 100%
2 — Answer quality & trust	Weeks 5-12	Trust before reach	Hybrid retrieval + rerank; citations; memory; abstention; feedback; TTS; playground	Golden-set scores improve; citation coverage ≥ 90%; feedback loop in use
3 — Distribution & enterprise	Weeks 13-26	Meet customers where they are	Website widget; tools + MCP; connectors; evaluation harness; enterprise pack	Widget live on a customer site; SSO pilot passes; nightly evals green

Table 1. Phase overview: windows, themes, outcomes, and exit criteria.

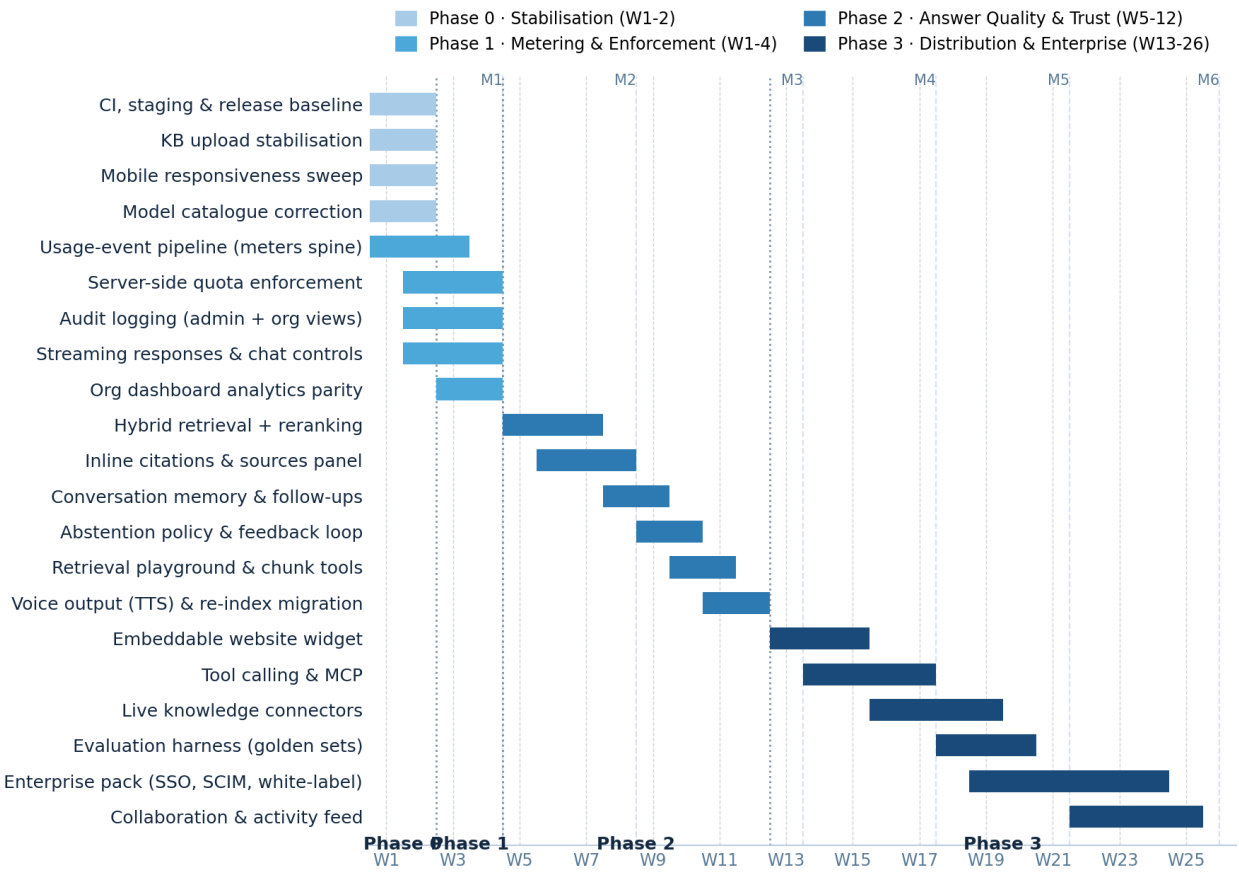


Figure 1. Implementation timeline across 26 weeks. Bars are workstreams grouped by phase; Phase 0 and Phase 1 deliberately overlap because they touch the same code paths.

> **How to read the rest of this document.** Chapters 2 through 5 specify each phase: goals, workstreams, schemas, algorithms, and acceptance criteria. Chapter 6 defines the cross-phase engineering standards that keep the phases coherent. Chapter 7 consolidates every acceptance test into one QA matrix, including the plan-by-plan quota validation the platform owner explicitly requested. Chapters 8 and 9 cover risk, rollout, and the measurable definition of stable.

2. Phase 0 — Stabilisation Sprint (Weeks 1-2)

Phase 0 exists because five defects were reported against the running product, and every one of them is either a trust risk or a revenue risk: knowledge base uploads fail intermittently, non-dashboard pages do not meet the mobile standard the dashboard set, the embedding model console offers a chat model where an embedding model belongs, the requests-today meter reads zero, and subscription limits are not enforced. The last two are architectural and therefore belong to Phase 1; the first three are fixable inside two weeks and are scheduled here. Phase 0 also installs the engineering baseline — continuous integration, a staging environment, and migration discipline — because every subsequent phase assumes a safe place to land code.

The sprint is intentionally overloaded with quick, high-visibility work. Closing the upload bug restores confidence in the core ingest path; the mobile sweep makes the whole product feel finished on phones, which is where most evaluation traffic arrives; and the model catalogue fix removes a trap that could silently corrupt retrieval for any organisation that touches it. All four workstreams ship behind ordinary pull requests with their acceptance tests attached, so the sprint doubles as the first full exercise of the new release pipeline.

2.1 WS-0.1 Engineering baseline (effort M)

Set up GitHub Actions to run lint, typecheck, unit tests, and a production build on every pull request, with the same pipeline gating pushes to main. Provision a staging environment that mirrors production configuration, including the database, object storage, model provider keys, and Razorpay test mode, so that acceptance tests exercise real integrations rather than mocks. Adopt migration discipline: every schema change ships as a forward-compatible Prisma migration, and migrations run as a deploy step, never by hand. Install structured logging and an error tracker (for example Sentry) with source-map upload, and create a shared feature-flag service so later phases can dark-launch safely. The baseline is finished when a broken pull request cannot merge, and a green pull request can be promoted to staging and then production with one command each.

- **Acceptance:** a deliberately failing test blocks merge in CI; a trivial change deploys to staging and production via pipeline only; an injected error appears in the tracker with stack trace and affected user within one minute.

2.2 WS-0.2 KB upload stabilisation (effort S)

The upload defect is diagnosed first, then fixed against the diagnosis. The likely root causes, in the order they should be investigated, are listed in Table 2. The fix specification follows a standard resumable-ingestion pattern: the client requests a presigned upload URL and uploads directly to object storage, which removes large multipart payloads from the application server entirely; a server-side handler then validates the object (size, MIME type by content sniffing rather than extension alone, and parser support) before enqueuing an ingestion job. Ingestion becomes an explicit queue with per-file status — queued, parsing, embedding, ready, failed-with-reason — surfaced in the KB detail view, and failures land in a retry queue that organisation owners can see and re-trigger rather than silently disappearing.

Symptom	Likely root cause	Fix	Verification
Upload times out or hangs on large files	Multipart body parsed in the route handler; platform body-size and function-timeout limits	Presigned direct-to-storage upload; route handles metadata only	50 MB PDF uploads and completes on a throttled 3G profile
Some file types fail silently	Client-side type check only; server accepts and drops unknown types later	Server-side MIME sniffing plus explicit allowlist; parser support matrix per format	A .exe renamed to .pdf is rejected with a clear message

Symptom	Likely root cause	Fix	Verification
Upload succeeds but document never appears	Parsing or embedding step fails after the upload resolves; no status is written	IngestionJob rows with per-file status; failure reason visible; retry action	Failing document shows "failed" with reason and a working retry
Duplicate documents after retry	Idempotency key absent on the ingest handler	Client-generated upload id; handler deduplicates on it	Double-clicked upload creates exactly one document

Table 2. KB upload defect: symptom-to-root-cause matrix with fixes and verification.

- **Acceptance:** uploading the same 50 MB PDF three times on a flaky connection produces exactly one ready document; unsupported types are rejected server-side with actionable copy; every upload visible in the UI has a machine-readable status; no ingestion failure is silent.

2.3 WS-0.3 Mobile responsiveness sweep (effort M)

The dashboard already meets the mobile standard and serves as the reference. Extract its effective rules into design tokens — a spacing scale, a type scale, a minimum interactive control height of 44 pixels, 16-pixel page gutters, and card gap minimums — and encode them once in the shared theme. Then sweep every remaining surface: the landing page first (it is the first thing a prospective customer sees on a phone), then auth screens, agent builder tabs, KB management, billing pages, and the platform admin console. The sweep is executed against a fixed device matrix (a small phone at 360 pixels, a standard phone at 390, a large phone at 430, and a tablet at 768) and verified with both screenshots and automated viewport tests, so regressions are caught by CI rather than by customers. Component-level fixes — oversized fonts, oversized inputs, missing card gaps — are corrected in the shared components, not per page, so the fix propagates everywhere at once.

- **Acceptance:** no horizontal scroll at 360 pixels on any route; every interactive element at least 44 by 44 pixels; Lighthouse mobile accessibility and layout-shift scores of 90 or better on the landing and top five dashboard routes; the device-matrix screenshots are attached to the pull request as review evidence.

2.4 WS-0.4 Model catalogue correction (effort S)

The model console currently permits selecting gpt-4o-2024-08-06 — a chat model identifier — as the embedding model, and warns about vector dimension matching without being able to enforce it. The correction splits the single model list into two catalogues: chat models (the gpt-5.6 family, gpt-4o variants) and embedding models (the text-embedding-3 family), each entry carrying its provider, context or token limits, cost per million tokens, and — for embedding models — the output dimension. Selections are constrained to the correct catalogue, and a dimension mismatch between the selected embedding model and a knowledge base becomes a hard, explained error rather than a warning. The full re-indexing workflow that safely re-embeds existing chunks when the embedding model changes is a Phase 2 workstream (WS-2.8); Phase 0 only makes bad states unrepresentable in the console.

- **Acceptance:** the embedding dropdown contains only valid embedding models; selecting a model whose dimension differs from a KB dimension is blocked with remediation copy that links to the re-index workflow documentation; the stored model configuration passes a validation script that fails the build if a chat model id appears in an embedding field.

2.5 Phase 0 summary

WS	Workstream	Effort	Chapter / AC	Dependencies
0.1	Engineering baseline: CI, staging, migrations, flags, error tracking	M	2.1	None
0.2	KB upload stabilisation: presigned flow, validation, ingest jobs, retry queue	S	2.2	0.1
0.3	Mobile responsiveness sweep with design tokens and device matrix	M	2.3	0.1
0.4	Model catalogue split with dimension metadata and mismatch guard	S	2.4	None

Table 3. Phase 0 workstream summary.

3. Phase 1 — Metering, Audit & Enforcement Core (Weeks 1-4)

Phase 1 fixes the platform's two architectural defects and lays the spine that every later feature hangs from. The requests-today meter reads zero because chat completions evidently never write usage events; the fix is not a patch on the counter but an append-only usage-event pipeline that records every billable model invocation. Subscription limits can be bypassed because enforcement lives in the interface rather than the API; the fix is a quota middleware that checks plan limits server-side before every billable action. Audit logging of user prompts and AI answers, which today exists nowhere, is built on the same event spine, giving platform admins a complete record and organisation dashboards the user-question visibility that was requested. Streaming responses and organisation analytics parity complete the phase, so that by week four the platform counts everything it does, refuses what plans do not allow, and shows both to the right people.

3.1 WS-1.1 The usage-event pipeline (effort M)

Every billable AI invocation — a chat turn, an embedding batch, a transcription, later a text-to-speech render — appends one immutable row to a UsageEvent table. The row records who, what, and how much: organisation, user, agent, knowledge base, the model called, prompt and completion token counts, latency, and an estimated cost computed from the model catalogue's price list. Writes happen in the same request

lifecycle that performs the model call, and rows are never updated or deleted; corrections are new compensating rows. Meters then stop being guessed state and become simple aggregations: requests-today is a count over today's window in the organisation's timezone, monthly usage is a count over the billing period anchored to the Razorpay renewal date, and storage is a sum over document bytes. A nightly rollup job pre-aggregates daily totals per organisation so dashboards read one small table instead of scanning the event log.

Prisma schema — UsageEvent (append-only; no update or delete paths are exposed)

```
model UsageEvent {
  id               String   @id @default(cuid())
  orgId            String
  userId           String
  kind             UsageKind        // CHAT | EMBEDDING | TTS | OCR
  agentId          String?
  kbId             String?
  model            String
  promptTokens     Int       @default(0)
  completionTokens Int       @default(0)
  latencyMs        Int       @default(0)
  costUsdMicros    BigInt    @default(0)   // integer micro-dollars
  billingPeriodId  String?
  createdAt        DateTime  @default(now())

  @@index([orgId, createdAt])
  @@index([orgId, kind, createdAt])
}
```

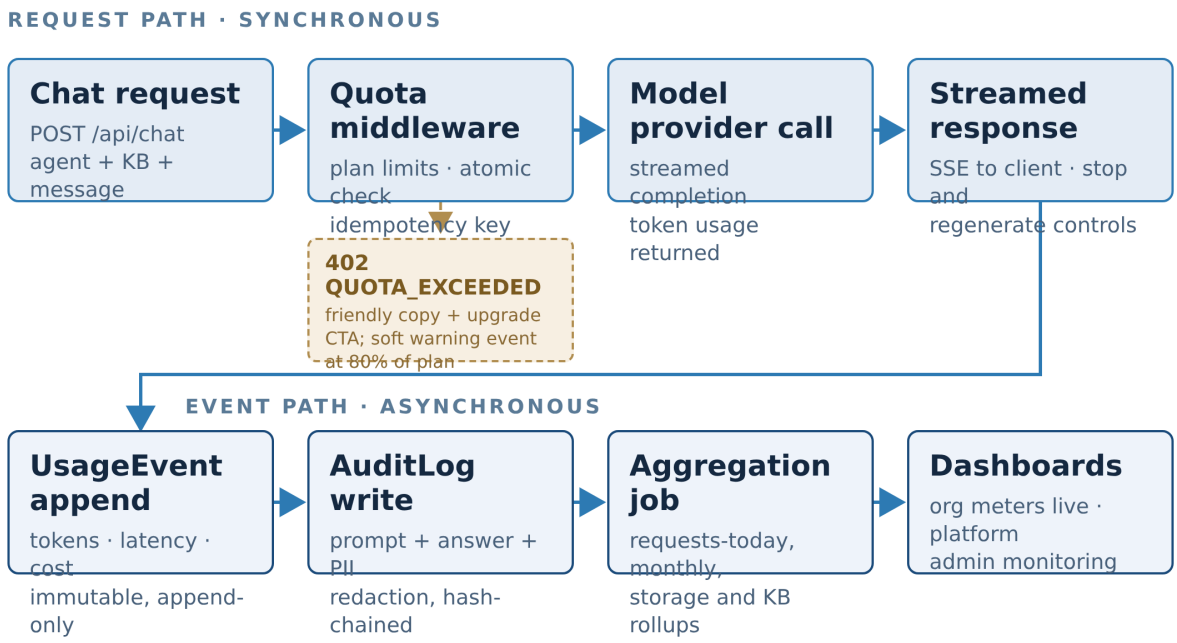


Figure 2. The usage-event spine: synchronous request path (top), asynchronous event path (bottom), and the quota middleware with its rejection branch.

- **Acceptance:** after one chat turn, exactly one CHAT UsageEvent exists with correct token counts and model name; the requests-today meter increments on the next dashboard poll; embedding and (later) TTS calls write their own event kinds; an event-audit script confirms zero update or delete statements ever executed against the table.

3.2 WS-1.2 Server-side quota enforcement (effort M)

A quota middleware wraps every billable route handler. It resolves the caller's organisation and plan, loads the plan's limit matrix, and checks each applicable limit atomically — either a transactional count against the usage-event window or a maintained counter incremented in the same transaction that admits the request, so concurrent requests cannot race past a limit. When a limit is exceeded, the API returns 402 with a machine-readable code, a friendly message, and an upgrade call-to-action payload; the client renders the upgrade sheet from that payload rather than from its own guess. At 80 percent utilisation, the middleware emits a soft-warning event that the client surfaces as a dismissible banner. Feature gates — model access, connectors, SSO — are checked in the same layer, because a capability hidden in the UI is not enforced; the API must refuse it too.

Reference implementation — quota middleware (TypeScript pseudocode)

```
export async function withQuota(req, plan, orgId) {
  // 1. feature gate: is this capability in the plan at all?
  if (!plan.features.includes(req.feature)) {
    throw new ApiError(403, 'PLAN_FEATURE_UNAVAILABLE', upgradeCTA(req.feature));
  }
  // 2. usage gate: atomic check-and-increment on the billing window
  const used = await db.$transaction(async (tx) => {
    const n = await countWindow(tx, orgId, req.limitKey, windowBounds(plan));
    if (n >= plan.limits[req.limitKey]) {
      throw new ApiError(402, 'QUOTA_EXCEEDED', upgradeCTA(req.limitKey));
    }
    if (n + 1 === Math.floor(0.8 * plan.limits[req.limitKey])) {
      emitSoftWarning(orgId, req.limitKey, n + 1);
    }
    return n + 1;
  });
  return used;
}
// Every billable handler: const used = await withQuota(...); then model call,
// then usageEvent append — the append is mandatory and sits in finally-style
// bookkeeping so failures still bill honestly if the provider charged.
```

Route / action	Limit checked	Limit key	On exceed
POST /api/chat (turn)	Requests per day and per month	requests_day, requests_month	402 + upgrade sheet; banner at 80%

Route / action	Limit checked	Limit key	On exceed
POST /api/kb (create)	Knowledge bases per org	kbs	402 + upgrade sheet; create button disabled when already at limit
POST /api/kb/:id/document	Storage bytes per org	storage_bytes	402 with remaining-storage copy; pre-upload client check
Model selection per agent	Plan model allowlist	feature:models	403 with plan comparison; models above tier hidden in UI
SSO, connectors, API keys	Plan feature flags	feature:sso, ...	403 with enterprise CTA; UI hides or locks the feature
Widget embed (Phase 3)	Plan widget flag and conversations	feature:widget, widget_convos	Widget renders an upgrade notice, never a silent failure

Table 4. Enforcement points: every billable or gated action, its limit, and its behaviour on exceed.

Two classes of bypass are explicitly closed. First, direct API calls: every enforcement point in Table 4 is testable with a raw HTTP client and no browser, and the acceptance tests do exactly that, because a quota that only holds when the UI behaves is not a quota. Second, concurrency: the check-and-increment runs inside a database transaction, so one hundred simultaneous requests against a plan with one remaining credit admit exactly one request and refuse ninety-nine. Rate limiting per user and per IP is layered above plan quotas as a separate abuse control, so a single user on an unlimited plan still cannot flood the platform.

3.3 WS-1.3 Plan-by-plan validation matrix (effort S)

The platform owner asked for each plan to be validated one by one, deeply, with blocking, warnings, error messages, and hiding all behaving properly. This workstream builds that validation as a permanent test suite rather than a one-off manual pass, so every future plan change is automatically re-verified. For each plan tier, a seeded organisation is created with its limits set near the boundary; scripted request loops then walk each limit to exactly N, N plus one, and beyond, asserting the refusal, the message copy, the meter accuracy, and the upgrade path at each step. UI-level tests assert the hiding and locking behaviour for plan-gated features, and API-level tests assert that hidden features are refused even when called directly.

Plan tier	Seeded state	Scripted validation	Expected behaviour
Free / trial	1 of 1 KB used; 49 of 50 requests today; 80% storage	Create 2nd KB; send requests 50-53; upload past storage; call premium model via raw API	KB create refused with upgrade CTA; request 51 refused; storage upload pre-checked and refused with remaining-bytes copy; premium model 403 even via API

Plan tier	Seeded state	Scripted validation	Expected behaviour
Starter	4 of 5 KBs; mid-month requests	Loop requests to monthly cap; verify daily reset at local midnight; verify 80% banner fired once	Monthly cap refuses with renewal-date copy; daily counter resets in org timezone; warning banner appears exactly at 80% and is dismissible
Pro / Business	Near all caps simultaneously	Parallel 100-request burst at 1 remaining; concurrent KB create at limit; meter reads vs event log	Exactly one admitted; refusals carry correct code and copy; meters equal event-log counts to the row; no negative or over-count
Enterprise	Unlimited flags on; SSO and custom LLM enabled	Exercise every enterprise feature; attempt feature below its plan via API to confirm gates are per-plan	No artificial caps encountered; feature gates still refuse capabilities the tier does not include; audit log records every action

Table 5. Plan-by-plan validation matrix: seeded state, scripted checks, and expected behaviour.

- **Acceptance:** the quota suite runs in CI against staging for every plan tier and is red on any bypass, any meter drift, or any missing upgrade path; a manual sign-off checklist per tier is attached to the release; the 81st request on a 80-request cap is refused with the standard error contract, from both the UI and a raw API call.

3.4 WS-1.4 Audit logging of prompts and answers (effort M)

Audit logging records every user prompt and every AI-generated answer as a first-class, immutable fact, visible to platform admins and — in a privacy-preserving, question-only form — on organisation dashboards. The AuditLog row captures the actor, organisation, agent, knowledge base, conversation, and message identifiers, the full prompt and answer text, the model, token counts, a redaction field listing what was masked, and a tamper-evident hash chain: each row hashes its content together with the previous row's hash, so any after-the-fact modification breaks the chain visibly. Writes share the transaction that persists the message, so an answer can never exist unlogged. A redaction layer masks emails, phone numbers, and payment identifiers by default; platform admins with a recorded justification can reveal the original, and the reveal itself is audited. Retention is tiered: ninety days hot and searchable, one year in cold archive, with organisation-configurable shortening but never lengthening past the compliance cap.

Prisma schema — AuditLog (hash-chained, append-only)

```
model AuditLog {
  id              String   @id @default(cuid())
  orgId           String
  actorUserId     String
  actorRole       String
  action          AuditAction    // PROMPT | ANSWER | CONFIG | BILLING | REVEAL
  agentId         String?
  kbId            String?
  conversationId  String?
  promptText      String?  @db.Text
  answerText      String?  @db.Text
  model           String?
  promptTokens    Int?
  completionTokens Int?
  redactions      Json      @default("[]")
  prevHash        String
  rowHash         String
  createdAt       DateTime  @default(now())

  @@index([orgId, createdAt])
  @@index([action, orgId, createdAt])
}
```

The platform admin console gains an audit viewer with filters by organisation, actor, action, agent, and time; CSV export for investigations; and a chain-verification action that recomputes the hash chain and reports integrity. The organisation dashboard gains the requested user-questions panel: a scoped, searchable list of the organisation's own questions with answer status, abstention flags, and feedback — the organisation-side view doubles as the content-gap backlog that Phase 2's abstention analytics feed.

- **Acceptance:** every chat turn produces exactly one PROMPT and one ANSWER audit row in the same transaction as the message; chain verification passes and fails loudly on a deliberate tamper test; redacted fields never render raw without a justification record; the org panel shows only the organisation's own questions; export produces valid CSV with the same filters.

3.5 WS-1.5 Streaming responses and chat controls (effort M)

Chat answers stream token-by-token from the provider through the API route to the client over server-sent events, with first-token latency budgeted at 1.5 seconds. The client gains the controls users now assume: a stop button that cancels generation and keeps the partial answer clearly labelled as truncated, a regenerate action that reuses the same context, per-message copy, and markdown rendering with syntax-highlighted code blocks behind a strict sanitiser. Streaming is also the enabling substrate for Phase 2's inline citations, which arrive as the answer streams rather than after it, so building it here is deliberate sequencing rather than polish. Usage events and audit rows are written on stream completion, including for cancelled streams, with a cancelled flag recorded.

- **Acceptance:** first token arrives under 1.5 seconds at p95 on a warm connection; stop cancels provider-side and the partial answer persists marked truncated; regenerate produces a fresh completion billed as a new turn; code blocks render highlighted and pass an XSS test suite; every completed or

cancelled stream writes its usage and audit records.

3.6 WS-1.6 Organisation dashboard analytics parity (effort S)

With the event spine live, the organisation dashboard's analytics view is rebuilt on real aggregations and brought to parity with the platform admin dashboard's monitoring. The org view shows requests today, the 7- and 30-day trend, active users, top questions, and per-agent activity, all scoped strictly to the organisation; the platform admin view adds cross-organisation totals, per-model latency percentiles, provider error rates, and ingestion queue depth. Dashboards poll on a sensible cadence with stale-while-revalidate semantics, so numbers feel live without hammering the rollup tables.

- **Acceptance:** requests-today matches a manual count of the event log for ten sampled organisations; org views cannot see other organisations' data under any filter combination; admin latency percentiles update within one minute of a spike injected in staging.

3.7 Phase 1 summary

WS	Workstream	Effort	AC section	Depends on
1.1	Usage-event pipeline and meters rebuild	M	3.1	0.1
1.2	Quota middleware: gates, 402 contract, soft warnings, race safety	M	3.2	1.1
1.3	Plan-by-plan validation suite (all tiers, UI + raw API)	S	3.3	1.2
1.4	Audit logging: hash chain, redaction, admin viewer, org questions panel	M	3.4	1.1
1.5	Streaming responses, stop / regenerate / copy, safe markdown	M	3.5	1.2
1.6	Org analytics parity and live meters	S	3.6	1.1

Table 6. Phase 1 workstream summary.

4. Phase 2 — Answer Quality & Trust (Weeks 5-12)

Phase 2 makes the assistant's answers measurably better and visibly trustworthy, and it builds the loop that keeps them improving after launch. The retrieval layer is upgraded from pure vector similarity to hybrid search with reranking, which the RAG literature consistently ranks as the highest-leverage quality change available to a production system. Answers gain inline citations that open the exact source passage, so verification is one click instead of one investigation. Conversations become first-class objects with memory, follow-up questions, and shareable history. The assistant learns to abstain honestly when the knowledge base cannot support an answer, and every abstention becomes a content-gap signal. Users gain thumbs feedback that routes unhappy conversations to a review queue. Knowledge-base owners get a retrieval playground to see exactly what the system would retrieve, and the embedding-model migration workflow makes model changes safe. Voice output completes the voice loop that voice input began.

4.1 WS-2.1 Hybrid retrieval and reranking (effort L)

Hybrid search combines two retrievers that fail in complementary ways. Dense vector search captures meaning but misses exact identifiers — product codes, error numbers, names that embed poorly; keyword search (BM25 over PostgreSQL full-text indexes) nails those exact tokens but misses paraphrase. Both run per query, their ranked lists are fused with Reciprocal Rank Fusion, and a second-stage cross-encoder reranker (a hosted reranking model or a self-hosted bge-family reranker) reorders the fused top-k into the final top-n that reaches the prompt. Query rewriting resolves pronouns and ellipses against the conversation history before retrieval runs, so "summarise that second point" retrieves the right documents; multi-query expansion optionally issues two or three reformulations for ambiguous questions and merges their results. The whole pipeline is behind a per-KB feature flag and evaluated on the golden set before default-on, because a retrieval change that is not measured is a regression waiting to ship.

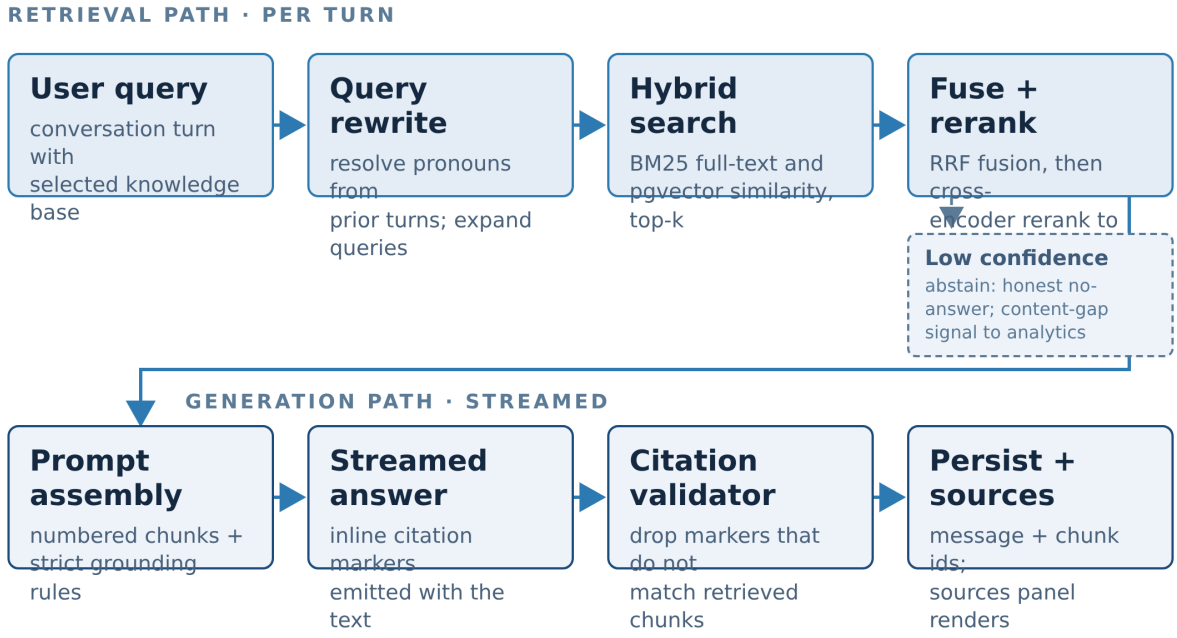


Figure 3. The Phase 2 retrieval and citation pipeline, with the abstention branch on low retrieval confidence.

Reference implementation — Reciprocal Rank Fusion (TypeScript pseudocode)

```
function rrfFuse(listA, listB, k = 60) {
  // listA: BM25 ranked chunk ids; listB: vector ranked chunk ids
  const scores = new Map();
  for (const [rank, id] of listA.entries()) {
    scores.set(id, (scores.get(id) ?? 0) + 1 / (k + rank + 1));
  }
  for (const [rank, id] of listB.entries()) {
    scores.set(id, (scores.get(id) ?? 0) + 1 / (k + rank + 1));
  }
  return [...scores.entries()]
    .sort((x, y) => y[1] - x[1])
    .map(([id]) => id);
}
// top-k per leg (k = 20) -> fuse -> rerank to top-n (n = 6) -> prompt context
```

- **Acceptance:** on the golden set, recall@10 improves by at least 15 percentage points over the vector-only baseline; identifier-heavy queries (codes, exact names) that failed before now retrieve their source; rerank adds under 300 milliseconds at p95; the pipeline flag can revert retrieval to vector-only in one configuration change without a deploy.

4.2 WS-2.2 Inline citations and the sources panel (effort M)

The retrieval layer already knows which chunks supported each answer; this workstream surfaces that knowledge to the user. Retrieved chunks are numbered in the prompt, and the system message instructs the model to place bracketed markers on the sentences each chunk supports. A post-processing validator then checks every marker against the set of chunk identifiers that was actually retrieved and silently drops any

fabricated reference — the guard against invented citations that the guardrails literature specifically warns about. In the UI, markers render as clickable chips; clicking opens a popover with the source document name, the exact passage, and a link to the document. A collapsible sources panel beneath the answer lists every contributing document. Persisted messages carry their chunk identifiers, so citations survive reloads and exports.

- **Acceptance:** on a 20-document golden KB, at least 90 percent of factual sentences carry a marker; every rendered marker resolves to a real stored passage (fabricated markers are dropped pre-render); the sources panel matches the persisted chunk set exactly; citation popovers meet the mobile touch-target standard.

4.3 WS-2.3 Conversation memory and controls (effort M)

Conversations become first-class persisted objects: a sidebar lists them with rename and pin; each conversation binds to a knowledge base (and optionally an agent); and turns carry over automatically with an explicit, configurable memory policy — a token budget for history rather than an unbounded transcript, with the oldest turns summarised out. Suggested follow-up questions are generated in the same completion call as the answer at negligible marginal cost, guiding users deeper into the knowledge base and measurably increasing engagement. Chat export and share links produce a read-only, citation-preserving snapshot for circulation inside the organisation.

- **Acceptance:** follow-up questions resolve correctly across at least five turns of pronoun references; memory policy caps history tokens as configured; sidebar operations work on mobile; shared links are read-only, org-scoped, and revocable.

4.4 WS-2.4 Abstention policy and no-answer analytics (effort S)

When fused retrieval confidence is low — below a per-KB threshold on the reranker's scores — the assistant abstains with an honest, templated response: it states that the knowledge base does not appear to contain the answer, shows the closest things it did find, and suggests either asking differently or requesting that the document be added. Abstention replaces the hallucination failure mode with an actionable gap, which is precisely the behaviour procurement reviewers probe for. Every abstention is recorded as an event, so the organisation dashboard's content-gap report ranks the questions the KB could not answer — a ranked backlog of what to ingest next, derived from real user intent rather than guesswork.

- **Acceptance:** on a KB that lacks the answer to golden-set unanswerable questions, the assistant abstains in at least 95 percent of cases instead of fabricating; abstention rate is a first-class metric on org and admin dashboards; the content-gap report aggregates per KB and per week.

4.5 WS-2.5 Feedback loop and review queue (effort M)

Every message gains thumbs up and down with an optional comment; feedback is stored with the full message and conversation context for one-click reproduction. Negative feedback enters a review queue visible to organisation owners: each entry shows the question, the answer, its citations, and the retrieval trace,

and offers the likely remedies — edit the source document, re-chunk it, adjust the agent prompt, or file a content gap. Aggregations surface quality trends per agent, per KB, and per organisation, and a weekly digest routes the worst offenders to the right owner. This loop is what separates platforms that improve after launch from platforms that decay, and it feeds the evaluation harness in Phase 3 with real failure cases.

- **Acceptance:** feedback is captured in under two clicks; the review queue reproduces the exact conversation and retrieval trace; thumbs-down rate and its trend appear on org and admin dashboards; a resolved review can be marked fixed with the remedy recorded.

4.6 WS-2.6 Voice output (TTS) with per-agent voices (effort M)

The agent builder's Voice tab gains real text-to-speech: a provider abstraction (the platform's default plus an OpenAI-compatible alternative) renders answers as streaming audio with a per-agent voice selection, and the chat surface gains a listen button with an auto-speak option. Every render writes a TTS usage event — the Phase 1 spine already carries the kind — so voice costs are metered, quota-checked, and audited exactly like chat turns. Cached audio for identical text and voice avoids double-billing regenerations.

- **Acceptance:** audio starts streaming under one second from button press; voice settings persist per agent and preview instantly in the builder; TTS turns respect plan quotas with the standard 402 contract; usage events and audit rows appear for every render.

4.7 WS-2.7 Retrieval playground and chunk inspection (effort M)

Knowledge-base owners gain a playground: type a question, see the exact chunks the retriever would fetch, their BM25 and vector scores, their fused rank, and whether each survived reranking into the prompt. The same view enables chunk-level management — inspect the chunks a document produced, delete bad chunks without deleting the document, and preview how a different chunking strategy would split a sample. This single tool converts KB quality work from guesswork into a measurable loop and directly attacks the leading cause of bad answers: dirty input data.

- **Acceptance:** playground results match production retrieval bit-for-bit for the same query and KB state; chunk deletion takes effect in production retrieval immediately; the re-chunk preview requires no write access to the KB.

4.8 WS-2.8 Embedding-model migration and re-index workflow (effort M)

Changing an embedding model without re-indexing silently corrupts retrieval for every existing chunk; this workflow makes the change safe and observable. An admin initiates a migration on a KB; a background job re-embeds all chunks with the new model into a parallel index; progress is visible per document with throughput and failure states; retrieval flips to the new index atomically only when the job completes; and a one-click rollback restores the previous index without data loss. The workflow records the embedding model identity and dimension on each KB, and queries refuse to run against a dimension-mismatched state — the enforcement point Phase 0's catalogue fix prepared.

- **Acceptance:** a migration on a 10,000-chunk KB completes with zero downtime and no window of mixed-dimension retrieval; a mid-migration cancellation leaves the KB fully on the old model; the golden-set score is re-measured automatically post-migration and reported.

4.9 Phase 2 summary

WS	Workstream	Effort	AC section	Depends on
2.1	Hybrid retrieval (BM25 + vectors), RRF fusion, reranking, query rewrite	L	4.1	1.1
2.2	Inline citations, validator, sources panel	M	4.2	2.1, 1.5
2.3	Conversation memory, follow-ups, export and share	M	4.3	1.5
2.4	Abstention policy and content-gap analytics	S	4.4	2.1
2.5	Feedback loop and review queue	M	4.5	2.2
2.6	Voice output (TTS) with per-agent voices	M	4.6	1.1
2.7	Retrieval playground and chunk inspection	M	4.7	2.1
2.8	Embedding migration and re-index workflow	M	4.8	0.4, 2.1

Table 7. Phase 2 workstream summary.

5. Phase 3 — Distribution, Agent Intelligence & Enterprise (Months 4-6)

Phase 3 takes a platform that is now metered, enforced, audited, and trustworthy, and puts it where the customers are. The embeddable widget turns Clara agents from a destination into infrastructure that lives on customers' own websites. Tool calling and the Model Context Protocol let agents act, not just converse, while every tool invocation is audited and permissioned. Live connectors keep knowledge bases synchronised with the systems that actually hold the truth — Drive folders, Confluence spaces, sitemaps. The evaluation harness makes quality regression impossible to ship silently, and the enterprise pack (SSO, SCIM, RBAC, data-processing paperwork, white-labelling) converts the Enterprise plan from a list of promises into a procurement-ready package.

5.1 WS-3.1 Embeddable website widget (effort M)

A one-line script tag renders any published agent on any website: per-agent embed keys, origin allowlist enforcement at the API, customisable colours, logo, position, and welcome message, and an optional pre-chat lead-capture form that gates or enriches the conversation. Widget conversations are quota-checked against the hosting organisation's plan, rate-limited per visitor as an abuse control, and fully audited like in-app conversations. The embed page in the agent builder shows the snippet, a live preview, per-domain analytics, and a revoke action that instantly disables a leaked key.

- **Acceptance:** embedding on a static test page with one script tag renders a working agent; requests from non-allowlisted origins are refused; widget conversations appear in meters, audit logs, and analytics indistinguishably from app conversations; key revocation takes effect within one minute.

5.2 WS-3.2 Tool calling and MCP support (effort L)

Agents gain the ability to call tools: a registry of typed function schemas (JSON Schema) that the model can invoke mid-conversation with arguments the platform validates before execution. The first built-ins are web search, a calculator, and a generic HTTP connector with credential storage; the same registry speaks MCP, so any standards-compliant MCP server becomes a tool source without custom integration. Tools are enabled per agent with role-gated permissions, every invocation is audited with its arguments and result, and tool use is a plan-gated feature enforced server-side. A human-approval mode requires a click before sensitive tools execute, and daily tool-call budgets per organisation cap runaway loops.

- **Acceptance:** an agent with the HTTP tool completes a task requiring an external call; MCP-hosted tools appear in the registry after entering a server URL; every tool call is audited and visible in the conversation transcript; budgets and plan gates refuse over-use with the standard error contract.

5.3 WS-3.3 Live knowledge connectors (effort L)

Static uploads age badly; connectors keep knowledge current. URL and sitemap ingestion with a configurable crawl depth bring websites in; OAuth integrations for Google Drive, Confluence, and Notion bring the systems where documents actually live. Every source syncs on a schedule, computes a content delta, and re-indexes only changed chunks — the document-versioning machinery from Phase 2 generalised to external sources. Sync health is visible per source with last-run status, and failures alert the organisation owner rather than silently staling.

- **Acceptance:** a changed Confluence page is searchable in updated form within one sync cycle; unchanged documents are not re-embedded (observable in usage events); a revoked OAuth token produces a visible disconnected state with re-auth instructions, not silent staleness.

5.4 WS-3.4 Evaluation harness (effort M)

The harness maintains golden sets — 50 to 200 real questions per KB with expected answer points, seeded from real conversations and failed reviews — and scores them with RAGAS-family metrics: faithfulness (is

the answer supported by retrieved context), answer relevancy, context precision, and context recall. Runs execute nightly and on demand, and every deployment that changes prompts, models, or retrieval parameters triggers a regression run whose results gate the release. Scores trend on dashboards per KB and per agent, and failed cases flow back into the golden set, so the suite grows with the product. This is the measurement rig that makes every Phase 2 quality claim durable rather than a one-time demo.

- **Acceptance:** nightly runs execute unattended and publish scores; a deliberately degraded retrieval change is caught and blocks release in CI; every failed case is reviewable with its full trace; the harness costs are metered as EMBEDDING and CHAT usage events like any other workload.

5.5 WS-3.5 Enterprise pack (effort L)

The Enterprise plan's advertised features become real, verifiable capabilities. SSO via SAML 2.0 and OIDC with just-in-time provisioning; SCIM for automated user lifecycle; organisation roles (owner, admin, builder, member) enforced at the API on every route; DPAs, security documentation, and a penetration test report as sales artefacts; and white-labelling with custom domains and branding removal. Audit logging from Phase 1 is the prerequisite — selling SSO without audit trails tends to fail procurement review, which is why this pack could not ship earlier.

- **Acceptance:** a pilot organisation authenticates via SAML with JIT user creation; SCIM deprovision disables access within one sync; role-gated routes refuse unauthorised calls with 403; a custom-domain tenant serves the widget with the tenant's branding and no platform marks; the penetration test's criticals and highs are closed before GA.

5.6 WS-3.6 Collaboration and activity feed (effort M)

Agents become team artefacts: co-editing with change history built on the existing draft-and-publish lifecycle, inline comments on agents and prompt versions with assignment, and a per-organisation activity feed showing what colleagues shipped, asked, or flagged. The weekly ritual this enables — review the feedback queue together, assign fixes, ship — is what makes the platform sticky inside an organisation and measurably reduces churn risk.

- **Acceptance:** two editors cannot silently clobber each other (conflict detection on publish); comment threads resolve with the fix linked; the activity feed is filterable per member and per artefact type.

5.7 Phase 3 summary

WS	Workstream	Effort	AC section	Depends on
3.1	Embeddable website widget with keys, theming, lead capture	M	5.1	1.2, 1.5
3.2	Tool calling, built-ins, MCP client, permissions, budgets	L	5.2	1.4

WS	Workstream	Effort	AC section	Depends on
3.3	Live connectors: crawler, Drive, Confluence, Notion, delta sync	L	5.3	2.8
3.4	Evaluation harness: golden sets, RAGAS metrics, release gating	M	5.4	2.1
3.5	Enterprise pack: SSO, SCIM, RBAC, DPA, white-label	L	5.5	1.4
3.6	Collaboration: co-editing, comments, activity feed	M	5.6	1.4

Table 8. Phase 3 workstream summary.

6. Shared Engineering Standards

The four phases stay coherent because every workstream obeys the same cross-cutting standards: one data model with consistent multi-tenant scoping, one API error contract, one observability stack, one security baseline, and one testing pyramid. This chapter defines those standards once so the phase chapters can reference them instead of repeating them. New engineers onboarded mid-programme should be able to read this chapter plus the current phase chapter and produce code that looks like it was written by the team from day one.

6.1 Data-model additions

The plan introduces a compact set of new models, each owned by exactly one workstream and each carrying orgId for tenant scoping where relevant. The catalogue below lists every new model, its purpose, and the phase that introduces it; the full column-level schemas for the two load-bearing tables (UsageEvent and AuditLog) appear in Chapter 3, and the remainder follow the same conventions: cuid identifiers, createdAt timestamps, orgId indexes on every query path, and no nullable column that the application cannot render a meaningful state for.

Model	Purpose	Introduced by	Key invariants
UsageEvent	One immutable row per billable AI invocation	WS-1.1	Append-only; indexed on (orgId, createdAt)
AuditLog	Immutable prompt and answer record with hash chain	WS-1.4	Append-only; rowHash links to prevHash
Conversation / Message	Persisted conversations with retrieval trace	WS-2.3	Message stores retrieved chunk ids
MessageFeedback	Thumbs and comment per message	WS-2.5	One per user per message, updatable

Model	Purpose	Introduced by	Key invariants
IngestionJob	Per-file ingest status with retry semantics	WS-0.2	Terminal states only via worker
DocumentVersion	Versioned documents with delta re-index	WS-3.3	Latest version is the indexed one
AgentTool / ToolCall	Tool registry and per-invocation records	WS-3.2	Every call audited with args and result
EmbedKey	Per-agent widget embed keys with origin allowlist	WS-3.1	Revocation is immediate
EvaluationRun / Case	Golden-set runs and scored cases	WS-3.4	Runs are immutable score sets
SsoConnection	Per-org SAML/OIDC and SCIM configuration	WS-3.5	One active connection per org

Table 9. New data models, their owning workstreams, and their invariants.

6.2 API conventions

All new endpoints follow one error contract so clients can render failures uniformly. Quota and feature refusals use 402 and 403 respectively with machine-readable codes and an upgrade payload; validation failures use 422 with a field-annotated payload; unexpected failures use 500 with a correlation identifier that also appears in the server log line. Every billable endpoint accepts an idempotency key so retries after network failures cannot double-charge a quota, and list endpoints paginate with stable cursors rather than offsets so quota suites and analytics consumers get consistent reads. Rate limits are communicated through standard headers, and the widget's public endpoints additionally validate origin and embed key before any per-visitor logic runs.

API error contract — quota example

```
// Standard error envelope (all routes)
{
  "error": {
    "code": "QUOTA_EXCEEDED",          // machine-readable
    "message": "Daily request limit reached on the Starter plan.",
    "upgrade": { "plan": "pro", "ctaUrl": "/billing/upgrade?reason=requests_day" },
    "correlationId": "01J8ZQ..."       // present on 5xx; matches server logs
  }
}
```

6.3 Observability

Structured JSON logging with a request correlation identifier is the floor; on top of it sit OpenTelemetry traces that follow one chat turn from the API route through retrieval, the model call, and the event append, so a slow answer decomposes into its parts on demand. The platform admin dashboard carries the operational panels: LLM latency percentiles per model, provider error and timeout rates, retrieval failure and abstention rates, ingestion queue depth, and quota refusal counts per plan. Alert thresholds page a human only on

user-visible degradation — p95 first-token latency above four seconds, provider error rate above five percent, or ingestion queue depth growing for fifteen minutes — because an alert that does not demand action trains the team to ignore alerts.

6.4 Security and privacy baseline

Multi-tenant isolation is enforced at the data layer: every query path includes orgId scoping, and a CI lint rule fails any raw database access that omits it. Secrets live in the platform secret store, never in the repository, and rotate on a schedule. The OWASP LLM Top 10 is the checklist for AI-specific risks: prompt-injection hardening (retrieved content is data, never instructions; tool calls are permissioned and audited), output filtering, and PII redaction on both prompt and answer sides of the audit log. Encryption in transit everywhere and at rest for storage and database volumes, and a penetration test before the enterprise pack goes generally available, close the baseline.

6.5 Testing pyramid

Unit tests cover services and pure functions; integration tests cover API routes against a real database and the provider abstraction in record-replay mode; end-to-end Playwright tests cover the user-visible flows of every phase, including the quota suites from WS-1.3 and the mobile device matrix from WS-0.3; and load tests exercise streaming under concurrency before each phase exit. The rule that keeps the pyramid honest is that every acceptance criterion in this document maps to at least one automated test that runs in CI, which is exactly what Chapter 7 tabulates.

7. Acceptance & QA Matrix

This chapter consolidates every acceptance criterion into one matrix, including explicit entries for the five defects the platform owner reported. It is the single source of truth for what done means, and the QA protocol that follows it defines how the matrix is executed: seeded organisations per plan tier, scripted loops that walk each limit to and past its boundary, raw-API bypass attempts without a browser, and a fixed mobile device matrix. The matrix is versioned with the code, and a phase's exit review walks its rows line by line.

ID	Capability / defect	Acceptance test	Sev.	Phase
D1	Requests-today meter reads zero	One chat turn; UsageEvent row exists; meter increments on next poll; equals event-log count	Blocker	1
D2	No audit log of prompts and answers	Prompt and answer audit rows written transactionally; admin viewer filters and exports; org panel shows own questions only	Blocker	1
D3	Usage limits not enforced / bypassable	Per-plan suites: N-th request admitted, (N+1)-th refused with 402 contract from UI and raw API; 100-way concurrent burst admits exactly one at the cap	Blocker	1

ID	Capability / defect	Acceptance test	Sev.	Phase
D4	KB uploads fail intermittently	Flaky-network triple upload yields one ready document; silent failures impossible; retry queue works	Blocker	0
D5	Non-dashboard pages below mobile standard	No horizontal scroll at 360px on any route; 44px targets; Lighthouse ≥ 90 on landing and top routes	Major	0
F1	Streaming with stop / regenerate / copy	p95 first token < 1.5s; stop truncates and persists; regenerate bills as new turn	Major	1
F2	Inline citations and sources panel	≥ 90% of factual sentences cited; all markers resolve to real passages; panel matches persisted chunks	Major	2
F3	Hybrid retrieval + reranking	Golden-set recall@10 +15pp vs baseline; identifier queries retrieve; rerank < 300ms p95	Major	2
F4	Abstention policy	≥ 95% honest abstention on unanswerable golden set; content-gap report aggregates	Major	2
F5	Feedback loop and review queue	Two-click capture; queue reproduces conversation and trace; trends on dashboards	Major	2
F6	Voice output (TTS)	Sub-second audio start; per-agent voices; quota-checked and audited like chat	Minor	2
F7	Website widget	One-line embed works; origin allowlist enforced; widget traffic metered and audited; revocation < 1 minute	Major	3
F8	Tool calling and MCP	HTTP tool task completes; MCP server tools register; every call audited; budgets enforced	Major	3
F9	Evaluation harness	Nightly runs publish scores; degraded retrieval blocks release; failed cases return to golden set	Major	3
F10	Enterprise pack	SAML SSO pilot with JIT; SCIM deprovision < 1 sync; RBAC 403s on unauthorised routes; pen-test criticals closed	Major	3

Table 10. Acceptance and QA matrix: defects D1-D5 and features F1-F10 with tests, severity, phase.

Severity gates are deliberately strict: a Blocker fails the phase exit outright, and a Major fails it unless a dated waiver with an owner and a fix window is recorded. The plan-by-plan quota suites from WS-1.3 run on every merge to main against staging, so D3 can never silently regress once closed. Mobile checks run as visual-regression snapshots in the device matrix, and the Lighthouse budget runs on the deploy pipeline for the landing page and the five highest-traffic dashboard routes, failing the deploy on regression rather than filing a ticket.

8. Risks, Dependencies & Rollout Strategy

Every plan of this length carries risks that are cheaper to name now than to discover in production. The register below lists the most consequential ones with their likelihood, impact, and the mitigation already embedded in the plan. Beyond the register, three structural choices de-risk the whole programme: every feature ships behind a flag and can be dark-launched to internal organisations first; every schema change is forward-compatible so rollback never requires data surgery; and the weekly release train means any given bad release is small by construction.

Risk	L	I	Mitigation embedded in the plan
Model-provider cost spike or pricing change	M	H	Per-model cost metadata (WS-0.4) feeds cost estimates on every usage event; daily per-org cost rollups alert at plan-relative thresholds
pgvector scale ceiling on large KBs	M	M	Hybrid search reduces dependence on brute-force ANN; index strategy reviewed at Phase 2 exit with a migration path to a dedicated vector engine if p95 retrieval exceeds budget
Razorpay webhook duplication breaks billing windows	L	H	Idempotency keys and event deduplication in the billing-period service; webhook replay tests in the quota suites
Re-index migration locks a large KB	M	M	Parallel-index build with atomic flip (WS-2.8); migration runs throttled; rollback is a pointer change
Streaming blocked or buffered by CDN/proxy layers	M	M	SSE compatibility tested end-to-end in staging behind the production CDN; fallback to chunked polling if a customer network breaks SSE
Enterprise procurement outpaces engineering	M	M	DPA and security documentation start in Phase 2 as a parallel paperwork track; pen test scheduled with lead time; SSO pilot scoped to one friendly organisation
Team capacity smaller than assumed	H	M	Phases are independently shippable; Phase 3 workstreams are ordered by revenue leverage so a capacity cut trims scope, never quality gates

Table 11. Risk register: likelihood (L) and impact (I) rated L/M/H with plan-embedded mitigations.

Rollout follows a fixed cadence. Each workstream merges behind a flag; the flag turns on for an internal dogfood organisation; acceptance tests run against staging with the flag on; and the flag goes to a canary percentage of production organisations before full rollout, with the error budget of the affected routes watched throughout. Database migrations deploy ahead of code, forward-compatible, so a rolled-back release never orphans a schema change. The team shape assumed throughout is two to four full-stack engineers plus part-time design support; where the plan says effort L, it means that workstream is the dominant load of its weeks and should not share a sprint with another L.

9. Success Metrics & the Definition of Stable

The programme is judged by the numbers its own Phase 1 spine produces, which closes the loop: the platform that could not count anything at the start ends as the platform that measures itself. The KPI table below defines, per phase, the metrics that prove the phase did what it existed to do. The closure curve in Figure 4 shows the cumulative programme: roughly sixty tracked items — every gap and defect from the analysis — closed in a deliberate order, with trust infrastructure first and reach last.

Phase	Primary KPIs	Target
0	Defect closure; CI discipline; mobile budget	D1-D5 closed or mitigated; 100% of merges via CI; zero horizontal scroll at 360px
1	Meter accuracy; quota integrity; audit coverage; stream latency	Meters = event log for sampled orgs; zero bypass in suites; 100% of turns audited; p95 first token < 1.5s
2	Retrieval quality; citation coverage; abstention honesty; feedback loop SLA	Recall@10 +15pp; ≥ 90% sentences cited; ≥ 95% abstention on unanswerables; reviews resolved < 5 working days
3	Distribution; evaluation discipline; enterprise readiness	≥ 10 live widget deployments; nightly evals green with release gating; SSO pilot signed off

Table 12. Phase-level KPIs and their targets.



Figure 4. Cumulative gaps closed across the programme: roughly 60 tracked items, sequenced trust-first.

9.1 The definition of stable

Stable is a checklist, not a feeling. The platform is stable when all five reported defects are closed with their acceptance tests automated; when every plan tier's limits are enforced server-side and verified by the per-plan suites on each merge; when every prompt and answer is audited, redacted, and tamper-evident;

when meters reconcile against the event log exactly for sampled organisations; when p95 first-token latency, provider error rate, and ingestion queue depth have stayed inside their budgets for thirty consecutive days; and when the nightly evaluation harness has gated releases for two weeks without a silent regression. At that point the platform counts what it does, refuses what plans forbid, proves what it answers, and watches itself — which is precisely what this programme was commissioned to achieve.

9.2 Beyond the six months

A short backlog is deliberately left for after the window rather than crammed into it: multi-language retrieval with language-aware chunking; a prompt A/B testing loop on top of the evaluation harness; agentic workflow orchestration beyond single tool calls; marketplace distribution of agent templates; and usage-based overage billing on top of the usage-event spine. Each of these becomes materially cheaper once Phase 3 lands, which is the sign of a well-sequenced foundation rather than an unfinished plan.

10. References

Anthropic. (2025). Model Context Protocol specification. Retrieved from modelcontextprotocol.io

Amazon Web Services. (2024). Amazon Bedrock Knowledge Bases: ingestion and sync patterns. AWS Documentation.

Es, S., James, J., Espinosa-Anke, L., & Schoot, H. (2023). RAGAS: Automated evaluation of retrieval augmented generation. arXiv preprint arXiv:2309.15217.

Gao, Y., Xiong, Y., Gao, X., et al. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

Microsoft. (2024). Azure OpenAI Service: embeddings and vector search best practices. Microsoft Learn Documentation.

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085.

OpenAI. (2024). Embeddings guide and text-embedding-3 model family. OpenAI Documentation.

OWASP Foundation. (2025). OWASP Top 10 for large language model applications. owasp.org.

Playwright. (2025). End-to-end testing and device emulation documentation. playwright.dev

PostgreSQL Global Development Group. (2024). Full text search and pgvector extension documentation. postgresql.org.

Sentry. (2025). Error monitoring and tracing documentation. sentry.io

AICPA. (2024). Trust Services Criteria for Security, Availability, and Confidentiality (SOC 2). aicpa.org

Yao, S., Zhao, J., Yu, D., et al. (2023). Tree of thoughts and tool-use in language model agents. NeurIPS 2023.

Zheng, L., Chiang, W. L., Sheng, Y., et al. (2024). Judgment on APIs: evaluation pipelines for LLM systems. NeurIPS 2024 Datasets and Benchmarks.